AIR Model Gateway

Visual design: components, wiring, integration and operations

air-model-gateway 0.1.0 · AIR platform · as of 2026-10-25 · printable A4 landscape

What it is

One HTTP service through which every AIR service reaches a language or embedding model: `POST /v1/generate` for text (optionally structured JSON, optionally streamed) and `POST /v1/embed` for vectors. Callers choose a **model class** (`local`, `auto`, `cloud`) or a **published embedding model** (`local-768-v1`), never a model name. The gateway owns everything else: which models serve each class, failover, retries, circuit breakers, authentication, permissions, rate limits, budgets, cost accounting, caching and observability.

2
model routes, plus capabilities, health, readiness, metrics

5
providers: Ollama, llama.cpp slot, Gemini, Anthropic, OpenAI

5
AIR consumers, each with its own token and grants

3
model classes; 2 published embedding models

0
runtime dependencies on other AIR services

4
environments: laptop, integration, kind, GKE

How to read this document

Each page has one diagram and short notes. A **numbered pin** in a diagram points to the note with the same number. File paths are relative to the repository root. Colours mean the same on every page:

- Consumers (AIR services)
- Gateway core: catalog, router
- Cloud models (billed)
- Observability and pricing
- GCP infrastructure
- API edge and control plane
- Local models (free, private)
- State and secrets (Redis, ledger)
- Configuration
- Errors returned to callers

Contents

1. System context
2. Components
3. Request flow
4. Model choice
5. Resilience
6. Streaming
7. Governance
8. API quick reference
9. GKE deployment
10. Delivery pipeline
11. Environments and gates
12. Observability and operations
13. Repository map and ownership

Where the detail lives

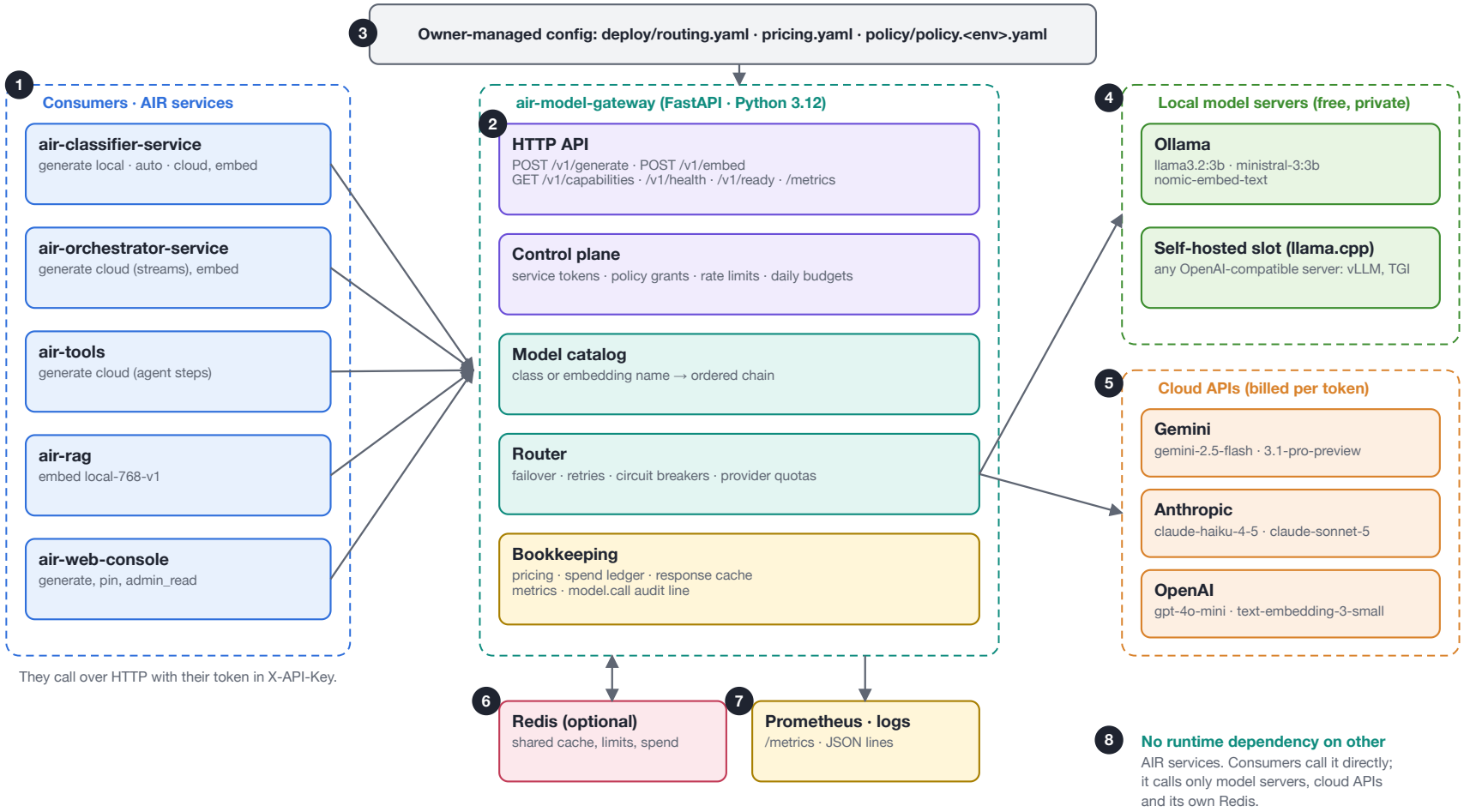
`docs/overview.md`	Team guide: every file, running, testing, troubleshooting
`docs/00-plan.md`	Scope, decisions, future work
`docs/01-hld.md`, `02-lld.md`	High- and low-level design (win on disagreement)
`docs/03-deployment.md`	GKE runbook for DevOps
`docs/04-integration.md`	Guide for teams calling the gateway
`docs/05-handover.md`	Assurance gates, risks, open items
`docs/openapi.json`	The wire contract (generate clients from it)

This document is generated from `docs/design/` with `make design`; when the system changes, change the source and regenerate.

1 · System context

Who calls the gateway, and what it calls

Big picture



1 Consumers
Five AIR services, each with its own token and grants. They call it directly over HTTP; there is no shared SDK requirement.

2 One API
Two model routes. `/v1/capabilities` tells a caller what its classes mean; health and readiness serve probes.

3 Owner-managed config
Models, chains, prices and grants are files in `deploy/`: an edit and a deploy, never a code release.

4 Local models
Free and private. `local` never leaves the network; `auto` tries local first.

5 Cloud models
Billed per token, priced from `deploy/pricing.yaml`. Cross-vendor chains survive one vendor's outage.

6 Redis, optional
Shares cache, rate limits and spend across replicas. Without it, or when it fails, each replica keeps state in memory.

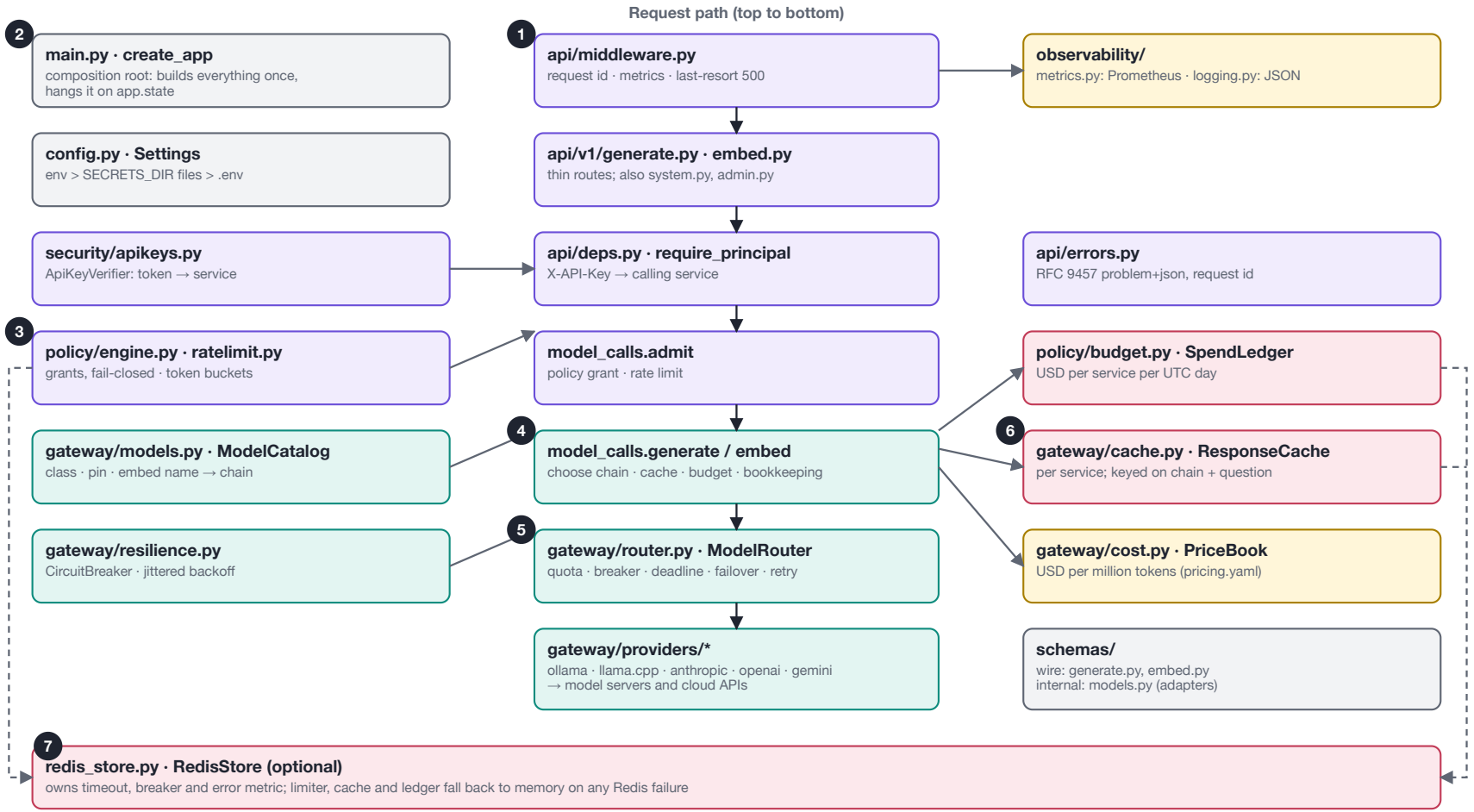
7 Observability
Prometheus metrics on `/metrics`; one JSON `model.call` audit line per served call.

8 Standalone
A hard design rule: the gateway needs no other AIR service at runtime, in any environment.

2 · Components

The modules under `src/air_model_gateway/` and how a request moves through them

Code



1 Thin edge
Middleware bounds the request id and counts requests; routes only authenticate and hand over to `api/v1/model_calls.py`, so the two model routes can't drift apart.

2 Composition root
`main.py::create_app` builds every collaborator once and hangs it on `app.state`. Outside `local` it refuses to start without its config files and tokens.

3 Control plane
Policy grants (fail closed outside local) and per-service token buckets.

4 The call path
`model_calls` chooses the chain (catalog), checks the cache and the budget, runs the router, then prices, records spend, stores the cache entry and writes the audit line.

5 Router
Runs a chain; it never decides which models. Adapters turn every failure into a `ProviderError`. Cloud SDKs load lazily.

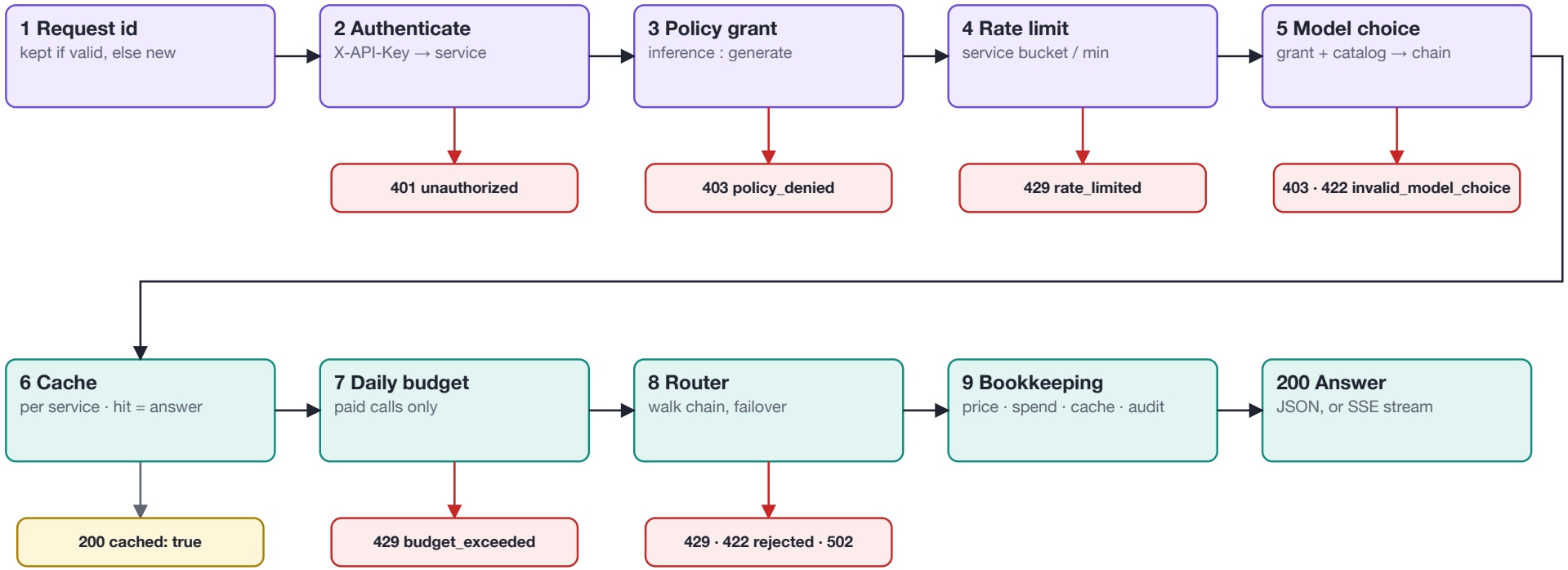
6 Per-service cache
Keyed on the calling service, the chain and every field that changes the question. Refusals and cut-off answers are never cached.

7 Degrade, never fail
All Redis access goes through `RedisStore`; each user falls back to memory. `/v1/ready` reports Redis but never gates on it.

3 · Request flow

`POST /v1/generate`, step by step, with every early exit

Runtime



POST /v1/embed takes the same path with action embed, an embedding name instead of a class, and no cache step.

Every error is application/problem+json with a stable code and the request id. Retry only 429 rate_limited, 502, 503.

Before the model (steps 1–7)

Everything cheap and local runs first: identity, grant, rate, choice, cache, budget. A request refused here costs no provider call. The key is checked before the body is read. Validation errors (unknown field, bad role, out-of-range `temperature`, over a size limit) are 422 and never reach a provider.

The model (step 8)

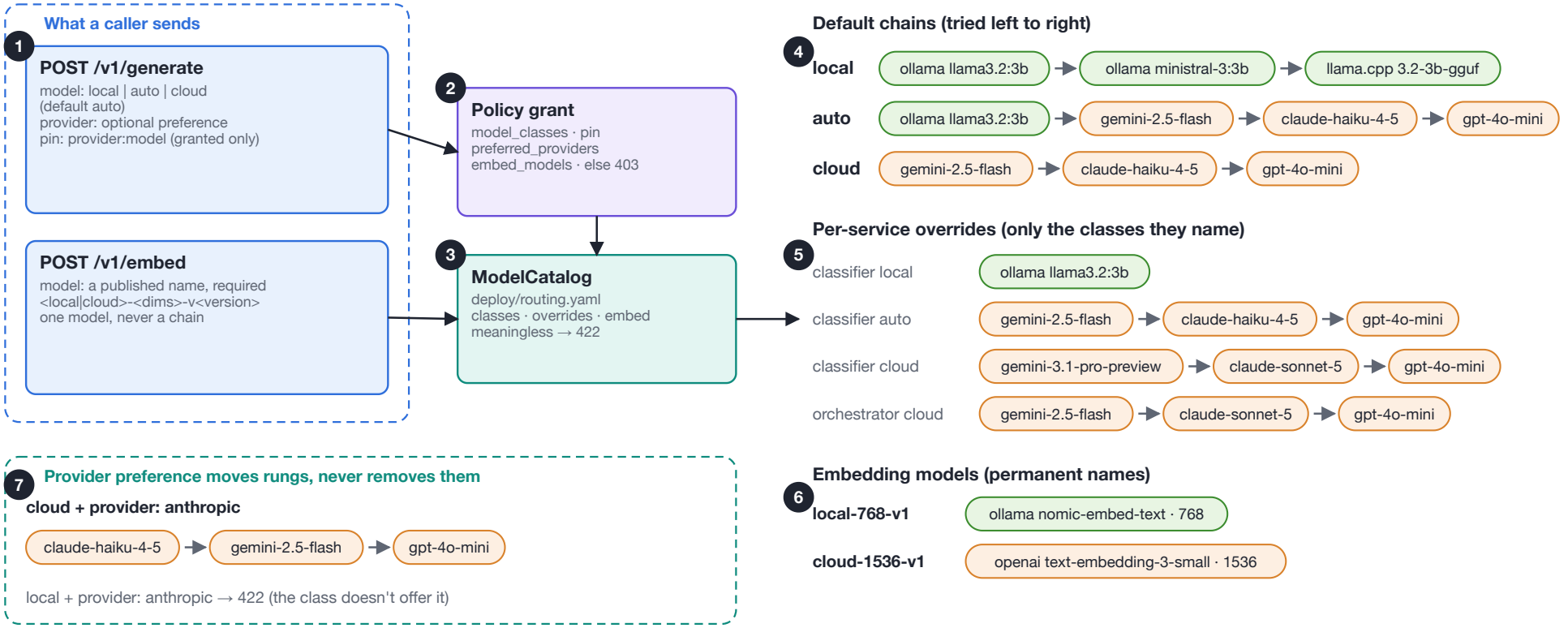
The router walks the chain the catalog chose: per-attempt deadline, open breakers and spent provider quotas skipped, one jittered retry if a failure was transient. It returns 429 only if every rung was throttled, 422 `provider_rejected` if every rung refused the request as invalid; otherwise 502 naming each rung's error.

After the model (step 9)

Price from `pricing.yaml` (local models are free), add to the service's daily spend, store in the cache, count tokens and cost, write one `model.call` line: service, operation, choice, provider, model, tokens, cost, cached, finish_reason.

4 · Model choice

Callers choose a class; `deploy/routing.yaml` decides the models



Rules enforced when routing.yaml loads: a local chain holds only local providers, a cloud chain only cloud ones; an embedding name states its location and dimensions, and the gateway checks every returned vector against them.

1 The request
`model` defaults to `auto`. `provider` is a preference, not a filter. `pin` names an exact model and stands alone.

2 Grant first
The policy says which classes, providers, embedding models and pins a service may use; outside the grant is 403.

3 Catalog
`gateway/models.py` turns an allowed choice into a chain. A choice that means nothing (unpublished name, provider not in the class) is 422.

4 Classes
`local` stays on our servers; `cloud` uses cloud models only; `auto` is local first with cloud fallback.

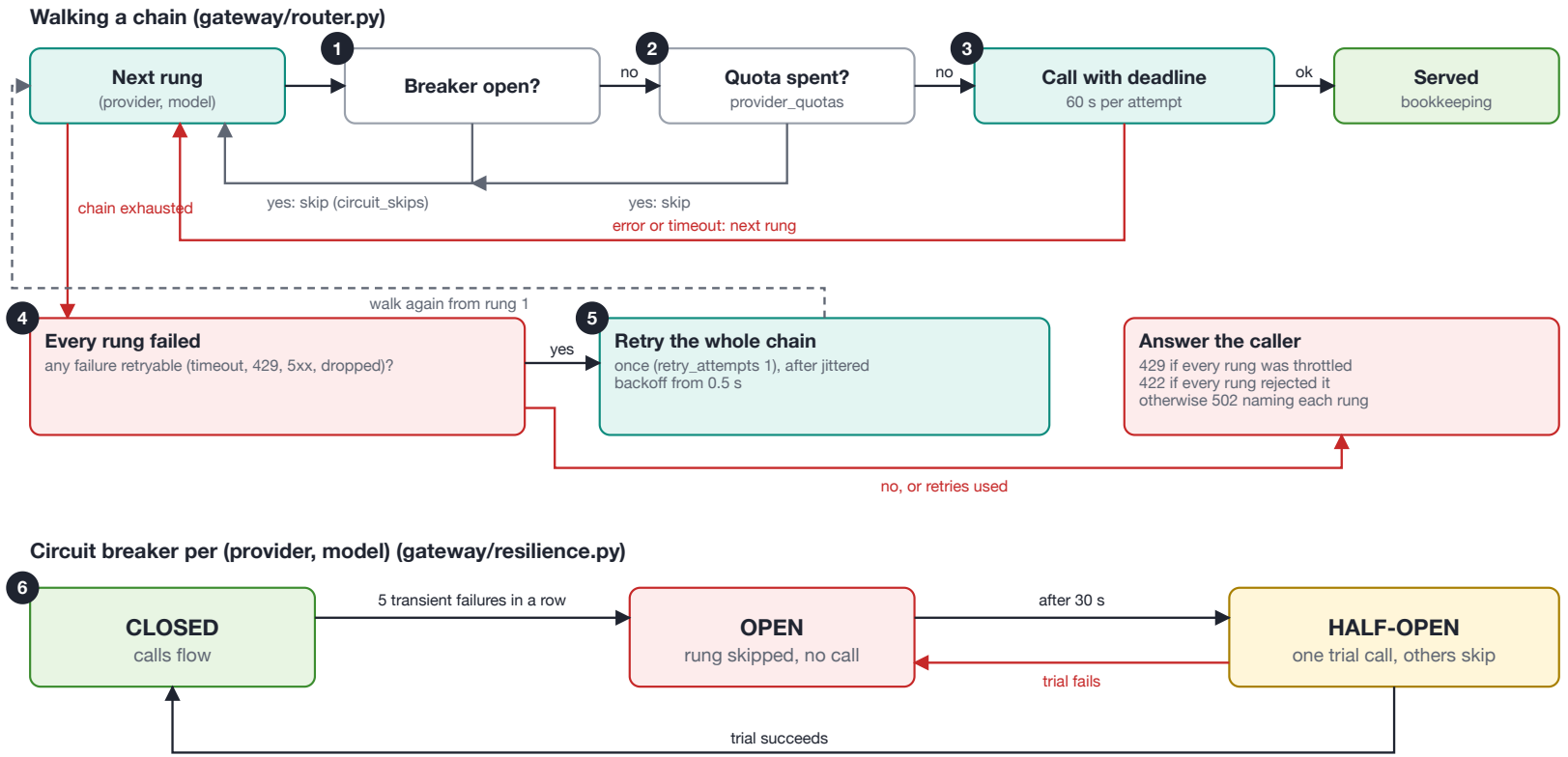
5 Overrides
A class can mean something different for one service; tuning one consumer never moves another. `/v1/capabilities` shows each service its own chains.

6 Embeddings
A name is permanent: one model, fixed dimensions, never a fallback. A new model is a new name (–v2) and a re-index.

7 Preference
Moves the provider's rungs to the front and keeps the rest as fallback. The cache key includes the chain, so a routing change invalidates old answers.

5 · Resilience

Failover, retries and circuit breakers in `gateway/router.py`



A 429 means the provider is up: it fails the rung but never counts toward the breaker. Provider SDKs run with max_retries=0, so the router is the only retry layer. Settings: gateway.request_timeout_seconds, retry_*, breaker_*.

1 Breaker check
An open breaker skips the rung without a call, so a dead model costs nothing while it is down.

2 Provider quota
`provider_quotas` caps calls per minute to a provider across all services; over quota, the rung is skipped and the chain fails over.

3 Deadline
Each attempt has its own deadline (60 s). A caller bounds the whole call with its own client timeout.

4 Retryable?
Timeouts, 429, 5xx and dropped connections are transient; a 400 or a refusal is not, and is never retried.

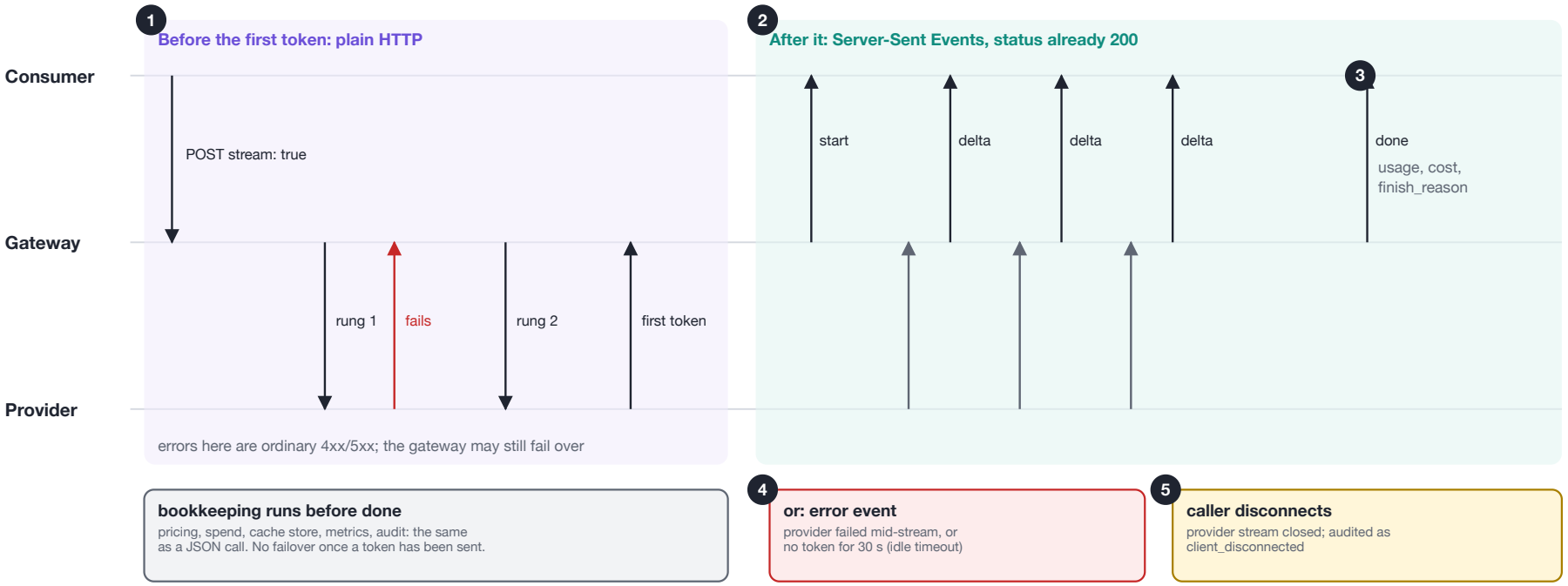
5 One retry layer
The router retries the whole chain once with jittered backoff. SDK retries are off so retries never multiply.

6 Breaker
Per (provider, model), per replica. Five consecutive transient failures open it; a trial call after 30 s closes or reopens it.

6 · Streaming

`stream: true` on `/v1/generate` answers with Server-Sent Events

Runtime



1 Before the first token
Everything can still fail as plain HTTP (401, 403, 422, 429, 502), and the router can still fail over to the next rung.

2 After it
Status is 200 and the model is fixed. Events: one `start`, many `delta`, then `done` or `error`.

3 done
The same summary as a JSON answer without `content`: id, model, provider, usage, cost, finish_reason.

4 error
A Problem body (code, detail, request id). The answer so far is incomplete; retry the whole call if needed.

5 Disconnect
The gateway closes the provider stream. The call is audited as `client_disconnected` and not billed to the budget (usage is unknown).

On the wire

```
event: start
data: {"id":"…","model":"llama3.2:3b","provider":"ollama","cached":false}

event: delta
data: {"content":"Hel"}

event: done
data: {"id":"…","usage":{…},"cost_usd":0.0,"finish_reason":"stop",…}
```

Headers `Cache-Control: no-cache` and `X-Accel-Buffering: no` keep proxies from buffering. A cache hit replays as one `delta`. Clients read with a read timeout above the 30 s idle limit, not a total timeout. `EventSource` can't POST: use `fetch` and read the body.

7 · Governance

Who may do what, how often, and for how much

Control plane

1

deploy/policy/policy.<env>.yaml

```
default_allow: false   # honoured only in local
default_rate_per_minute: 60
provider_quotas:       # all services
  gemini: 600          # calls/minute
services:
  air-rag:
    resources:
      inference: [embed]
    embed_models: [local-768-v1]
    rate_per_minute: 600
    daily_budget_usd: 25.0
  air-web-console:
    resources:
      inference: [generate]
      admin: [admin_read]
    pin: true
```

Unlisted fields mean "all": model_classes, embed_models and preferred_providers.

Typed at load: an unknown action or class stops startup. Quota and budget values are examples; the shipped files set none yet.

2

3

Production grants (policy.prod.yaml; staging is the same)

	generate	embed	admin_read	classes	embed models	pin	rate / min
air-classifier-service	✓	✓	—	all	all	—	300
air-orchestrator-service	✓	✓	—	all	all	—	600
air-tools	✓	—	—	all	—	—	300
air-rag	—	✓	—	—	local-768-v1	—	600
air-web-console	✓	—	✓	all	—	✓	60

4

Fails closed	Outside local, a service not listed gets nothing (403).
Rate limit	Token bucket per service, shared across replicas through Redis.
Provider quota	Shared by all services; a spent quota skips that rung.
Daily budget	USD per UTC day; then 429 budget_exceeded + Retry-After.
Tokens	X-API-Key per service, from Secret Manager; rotation keeps old + new.

1 **One file per environment**
`policy.local.yaml`, `policy.staging.yaml`, `policy.prod.yaml`. Read at startup, shown read-only at `GET /v1/admin/policy`.

2 **Per-service entry**
Resources and actions (`inference: generate, embed`; `admin: admin_read`), rate, model choices, pin, budget.

3 **Grants today**
Pinned by `tests/test_gateway_config.py`: dropping a grant a consumer relies on fails the build.

4 **Adding a consumer**
All three policy files, the test's consumer table, the token scripts, the smoke tests, Terraform's `consumers` and the GKE NetworkPolicy namespaces.

8 · API quick reference

The contract consumers code against: `docs/openapi.json`

Endpoints

Route	Purpose
`POST /v1/generate`	Text from messages; JSON or SSE
`POST /v1/embed`	One vector per input text
`GET /v1/capabilities`	Your classes, chains, grants (`allowed`), what this deployment can serve (`available`)
`GET /v1/health`	Liveness (no auth)
`GET /v1/ready`	200 if any provider answers, else 503
`GET /v1/admin/policy`	Effective policy (admin_read)
`GET /metrics`	Prometheus exposition

Headers

`X-API-Key`	Your service token (required)
`X-Request-ID`	Optional, ≤128 of `A-Za-z0-9._:-`; echoed and logged
`Retry-After`	On 429 `budget_exceeded`

Reference client

`src/air_model_gateway_client`: `generate`, `generate_stream`, `embed`; reads `AIR_MODEL_GATEWAY_URL` and `AIR_MODEL_GATEWAY_TOKEN`.

Generate

```
POST /v1/generate
{
  "model": "auto",          // local | auto | cloud
  "provider": "anthropic",  // optional preference
  "messages": [
    {"role": "system", "content": "…"},
    {"role": "user", "content": "…"}
  ],
  "max_tokens": 300,
  "temperature": 0.2,
  "json_schema": {…},       // optional
  "schema_name": "verdict",
  "cache_prefix": true,
  "stream": false,
  "cache": true
}
```

Stateless: send the whole history each call. Unknown fields are 422.

```
200: id, content, model_class, model,
     provider, usage, cost_usd, cached,
     refusal, finish_reason,
     provider_finish_reason
```

Embed

```
POST /v1/embed
{"model": "local-768-v1",
 "input": ["text a", "text b"]}

200: id, embedding_model, dimensions,
     model, provider, embeddings,
     usage, cost_usd
```

Errors (application/problem+json)

Status · code	Meaning · action
401 `unauthorized`	Missing or unknown token
403 `policy_denied`	Not granted (operation, class, provider, pin, embedding model)
422 `validation_error`	Body invalid or over a size limit; detail names each field
404 · 405 `not_found` · `method_not_allowed`	Wrong path or method
422 `invalid_model_choice`	Choice means nothing here (or an unpriced pin); see capabilities
422 `provider_rejected`	Every provider refused the request as invalid; change it
429 `rate_limited`	Retry with backoff
429 `budget_exceeded`	Stop until `Retry-After` (next UTC day)
502 `provider_unavailable`	Every rung failed; retry with backoff
503	Not ready; retry
500 `internal_error`	Bug; report the request id

finish_reason

`stop`, `length` (cut off at `max_tokens`: treat structured output as invalid), `refusal`, `other`, the same for every provider.

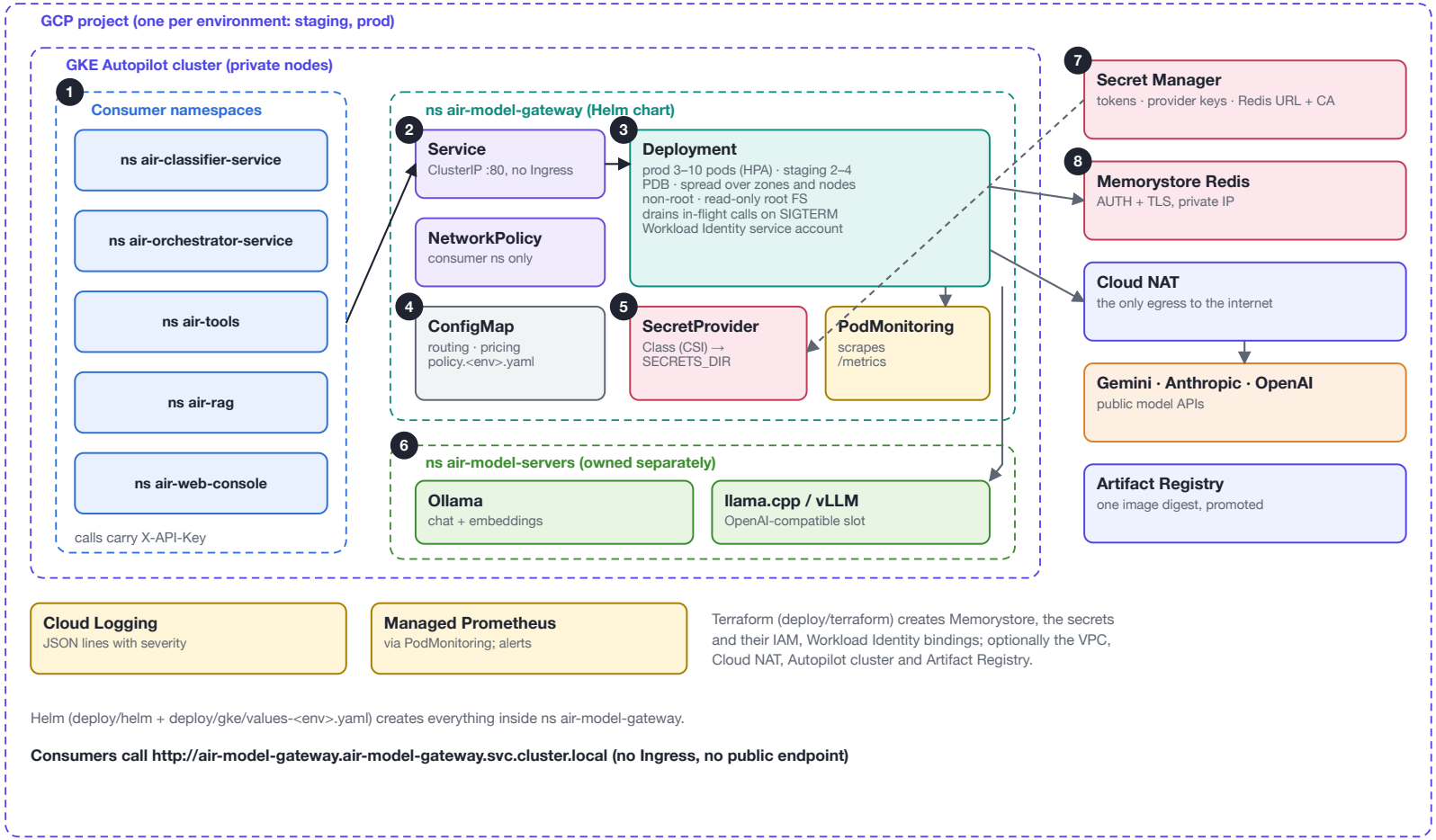
Cost

`cost_usd` is 0 for local models and cache hits. Prices are USD per million tokens.

9 · GKE deployment

Staging and production on GCP: in-cluster only

DevOps



1 Consumers
Pods in the listed namespaces only; the NetworkPolicy refuses everyone else.

2 Service
ClusterIP. No Ingress and no public load balancer.

3 Deployment
Hardened pods, HPA, PDB, zone spread; `preStop` plus a grace period lets in-flight calls finish.

4 Config
Routing, pricing and policy passed with `--set-file`; a change rolls the pods without a new image.

5 Secrets
Secret Manager mounted by the GKE add-on (CSI) as files in `SECRETS_DIR`; nothing secret in Kubernetes or GitHub.

6 Model servers
Run in their own namespace, owned separately; the gateway only gets their URLs from values.

7 Secret Manager
Consumer tokens, provider keys, Redis URL and CA. `deploy/gke/bootstrap-secrets.sh` loads and rotates them.

8 Memorystore
Shared cache, rate limits and spend across pods, with AUTH and TLS on a private IP.

10 · Delivery pipeline

From a commit to production, and the one-time setup per environment

DevOps

Every push and pull request: .github/workflows/ci.yml (= make verify)

1

check
make check: ruff, mypy strict, tests ≥ 90%
lock-check · OpenAPI drift

supply-chain
pip-audit · shellcheck · hadolint
docker build

deploy-config
helm lint · kubeconform
terraform validate (make gke-validate)

Tag v* (or manual): .github/workflows/deploy.yml

2 **Build once**
push image to staging
Artifact Registry

→ **Staging**
helm upgrade (gke-deploy)
then gke-smoke

→ 3 **Approval**
prod environment needs
a reviewer

→ **Promote**
copy the same digest
into prod's registry

→ **Prod**
helm upgrade
then gke-smoke

CI authenticates with Workload Identity Federation: no keys are stored in GitHub.

Once per environment (DevOps)

4 **terraform apply**
deploy/terraform/environments/<env>
modules: air-model-gateway, gke-platform

→ 5 **bootstrap-secrets.sh**
issue consumer tokens, load provider
keys; --rotate / --retire-old

→ **values-<env>.yaml**
fill placeholders: project, model-server
URLs, consumer namespaces

→ **Each consumer**
mounts its token, sets the gateway URL
(03-deployment.md §4.4)

Config change (routing, pricing, policy): edit the file, merge, deploy. The ConfigMap changes, the pods roll; no new image.

1 **CI = make verify**
The same gates run on a laptop and in CI, so a green laptop predicts a green build.

2 **Build once**
One image per release, pushed to staging's registry and tested there.

3 **Human approval**
The `prod` GitHub environment requires a reviewer; promotion copies the exact digest staging tested.

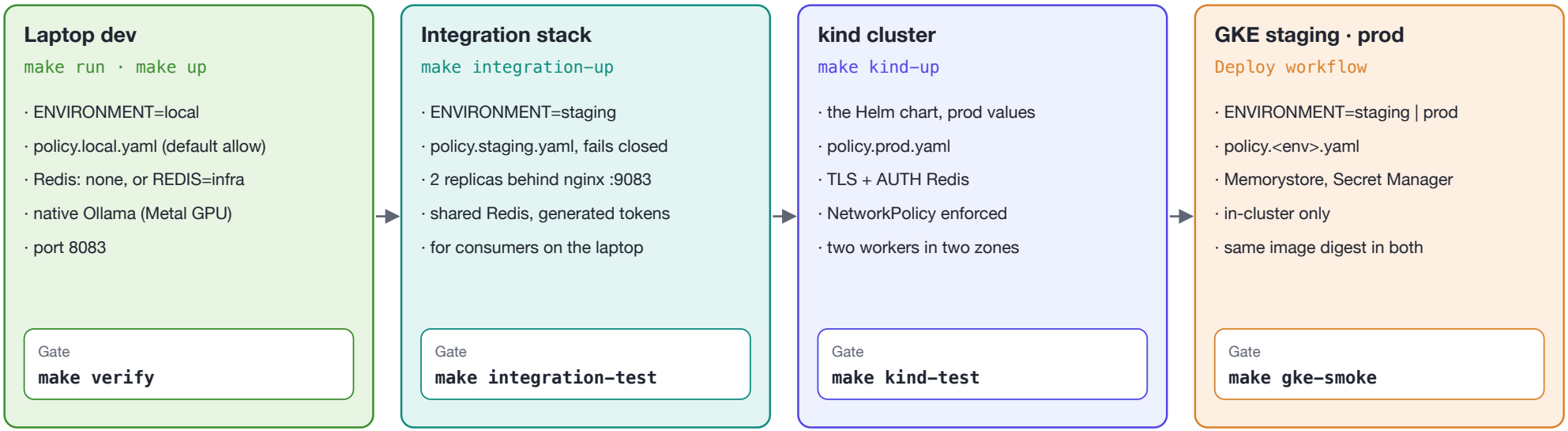
4 **Terraform**
Memorystore, secrets and IAM, Workload Identity; optionally the VPC, NAT, Autopilot cluster and registry.

5 **Tokens**
One per consumer, issued into Secret Manager. Rotation is two-phase (`--rotate`, then `--retire-old`) with no downtime.

11 · Environments and gates

Where it runs, and the command that proves each one

Assurance



Definition of done: make verify (what CI runs). A change to runtime behaviour, deployment or a consumer-visible contract also passes make kind-up kind-test and make integration-up integration-test.

make verify

Lint, mypy strict, tests with a 90% coverage floor, OpenAPI drift, lock freshness, dependency audit, shellcheck, hadolint, chart and Terraform validation. Exactly what CI runs.

Runtime gates

`integration-test` and `kind-test` make real calls per grant: 401 for an unknown token, 200/403 per grant, pins, embedding names, streaming, readiness, Redis, and (kind) the NetworkPolicy block. `gke-smoke` runs the same checks inside GKE after each deploy.

Settings

Prefix `AIR_MODEL_GATEWAY_`, `__` for nesting (`AIR_MODEL_GATEWAY_OLLAMA__BASE_URL`). Env beats `SECRETS_DIR` files beats `.env`. `.env.example` documents every key.

12 · Observability and operations

What to watch, and what to do when it moves

Operations

Metrics (prefix `air_model_gateway_`)

`requests_total`	route, method, status
`model_tokens_total`	provider, model, kind
`model_cost_usd_total`	provider, model
`service_cost_usd_total`	by calling service
`model_unpriced_total`	model missing from pricing
`provider_timeouts_total`	attempts past the deadline
`adapter_errors_total`	adapter bugs hidden by failover
`chain_retries_total`	extra passes over a chain
`circuit_skips_total`	rungs skipped, breaker open
`circuit_open`	1 while a breaker is open
`provider_quota_rejected_total`	rungs skipped, quota spent
`ratelimited_total`	requests over the rate
`budget_rejected_total`	requests over the budget
`redis_errors_total`	degraded to memory
`streams_total`	completed, error, client_disconnected

Audit line (one per served call)

```
{
  "event": "model.call",
  "service": "air-rag",
  "operation": "embed",
  "model_choice": "local-768-v1",
  "provider": "ollama",
  "model": "nomic-embed-text",
  "tokens": 42, "cost_usd": 0.0,
  "cached": false, "finish_reason": null,
  "request_id": "…", "severity": "INFO"
}
```

JSON lines with a Cloud Logging severity. Filter on `service`, `model` or `request_id`; a caller's `X-Request-ID` ties its logs to the gateway's.

Alerts to create first

`circuit_open == 1` for 5 min	A model is down
rising `redis_errors_total`	Running degraded
`model_unpriced_total > 0`	A model has no price
`adapter_errors_total > 0`	An adapter bug (often an SDK change)
5xx share of `requests_total`	Chains exhausted

Troubleshooting

Symptom	Look at
502 naming every rung	Model servers up? Keys set? `/v1/ready`, breaker metrics
403 on a call	The service's entry in `policy.<env>.yaml`; `/v1/capabilities`
422 `invalid_model_choice`	Name not published, provider not in the class, pin unpriced or unknown
429 `budget_exceeded`	`daily_budget_usd`; resets at 00:00 UTC
Slow answers	`provider_timeouts_total`; a slow first rung; Ollama loaded at the right context
Answers changed after a deploy	The class's chain in `routing.yaml`; tune one service with `overrides`
Pod not ready	No provider answers; Redis never blocks readiness

Full table: `docs/overview.md` §9. Runbook: `docs/03-deployment.md` §5.

13 · Repository map and ownership

Where things live, and who changes them

Layout

```
air-model-gateway/
├─ src/air_model_gateway/
│  ├─ main.py          composition root
│  ├─ config.py        settings
│  ├─ api/             middleware, routes,
│  │                   model_calls, errors
│  ├─ gateway/         catalog, router,
│  │  │                resilience, cache, cost
│  │  └─ providers/    one adapter each
│  ├─ policy/          engine, ratelimit, budget
│  ├─ security/        service tokens
│  ├─ schemas/         wire + internal contracts
│  └─ observability/   metrics, logging
├─ src/air_model_gateway_client/
├─ tests/              one file per concern
├─ deploy/
│  ├─ routing.yaml  pricing.yaml  policy/
│  ├─ helm/  gke/  terraform/
│  └─ local/  integration, kind, prometheus
├─ docs/               PDF, guides, contract
└─ Makefile  Dockerfile  docker-compose.yml
```

Who changes what

Change	Where · how
Which models serve a class	`deploy/routing.yaml` · edit, deploy
A price	`deploy/pricing.yaml` · edit, deploy
A grant, rate, budget, quota	`deploy/policy/policy.<env>.yaml` · edit, deploy
A new embedding model	New name in `routing.yaml`; consumers re-index
Infrastructure	`deploy/terraform` · plan, apply
Chart or values	`deploy/helm`, `deploy/gke` · `make gke-validate kind-up kind-test`
Wire contract	`schemas/` · `make openapi`, client, integration guide
A provider	Adapter, `Provider` member, wiring, prices, tests
A dependency	`pyproject.toml` · `make lock verify`

Checklists for each: `AGENTS.md` "Change checklists".

Design principles

Standalone	No other AIR service at runtime.
Classes, not models	Callers state intent; the owner picks models.
Config, not code	Models, prices and grants change without a release.
Fail closed	Outside local, nothing is allowed unless granted.
Degrade, never fail	Redis is optional; every user falls back.
One retry layer	The router; SDK retries are off.
Stateless	No conversations kept; callers own history.
One contract	`docs/openapi.json`, checked against the code.

Regenerating this document

`make design` redraws `docs/design/diagrams/*.svg` from `diagrams.py` and prints `docs/design/design.html` to `docs/air-model-gateway-design.pdf` (needs Google Chrome).